Add native code as a plugin

The easiest way to provide an AI agent with capabilities that are not natively supported is to wrap native code into a plugin. This allows you to leverage your existing skills as an app developer to extend the capabilities of your AI agents.

Behind the scenes, Semantic Kernel will then use the descriptions you provide, along with reflection, to semantically describe the plugin to the AI agent. This allows the AI agent to understand the capabilities of the plugin and how to interact with it.

Providing the LLM with the right information

When authoring a plugin, you need to provide the AI agent with the right information to understand the capabilities of the plugin and its functions. This includes:

- The name of the plugin
- The names of the functions
- The descriptions of the functions
- The parameters of the functions
- The schema of the parameters
- The schema of the return value

The value of Semantic Kernel is that it can automatically generate most of this information from the code itself. As a developer, this just means that you must provide the semantic descriptions of the functions and parameters so the AI agent can understand them. If you properly comment and annotate your code, however, you likely already have this information on hand.

Below, we'll walk through the two different ways of providing your AI agent with native code and how to provide this semantic information.

Defining a plugin using a class

The easiest way to create a native plugin is to start with a class and then add methods annotated with the `KernelFunction` attribute. It is also recommended to liberally use the `Description` annotation to provide the AI agent with the necessary information to understand the function.

> **Tip**

The following `LightsPlugin` uses the `LightModel` defined here.

C#

```
public class LightsPlugin
{
   private readonly List<LightModel> _lights;

   public LightsPlugin(LoggerFactory loggerFactory, List<LightModel> lights)
   {
      _lights = lights;
   }

   [KernelFunction("get_lights")]
   [Description("Gets a list of lights and their current state")]
   public async Task<List<LightModel>> GetLightsAsync()
   {
      return _lights;
   }

   [KernelFunction("change_state")]
   [Description("Changes the state of the light")]
   public async Task<LightModel?> ChangeStateAsync(LightModel changeState)
   {
      // Find the light to change
      var light = _lights.FirstOrDefault(l => l.Id == changeState.Id);

      // If the light does not exist, return null
      if (light == null)
      {
         return null;
      }

      // Update the light state
      light.IsOn = changeState.IsOn;
      light.Brightness = changeState.Brightness;
      light.Color = changeState.Color;

      return light;
   }
}
```

Tip

Because the LLMs are predominantly trained on Python code, it is recommended to use snake_case for function names and parameters (even if you're using C# or Java). This will help the AI agent better understand the function and its parameters.

> **Tip**
>
> Your functions can specify `Kernel`, `KernelArguments`, `ILoggerFactory`, `ILogger`, `IAIServiceSelector`, `CultureInfo`, `IFormatProvider`, `CancellationToken` as parameters and these will not be advertised to the LLM and will be automatically set when the function is called. If you rely on `KernelArguments` instead of explicit input arguments then your code will be responsible for performing type conversions.

If your function has a complex object as an input variable, Semantic Kernel will also generate a schema for that object and pass it to the AI agent. Similar to functions, you should provide `Description` annotations for properties that are non-obvious to the AI. Below is the definition for the `LightState` class and the `Brightness` enum.

C#

```
using System.Text.Json.Serialization;

public class LightModel
{
    [JsonPropertyName("id")]
    public int Id { get; set; }

    [JsonPropertyName("name")]
    public string? Name { get; set; }

    [JsonPropertyName("is_on")]
    public bool? IsOn { get; set; }

    [JsonPropertyName("brightness")]
    public Brightness? Brightness { get; set; }

    [JsonPropertyName("color")]
    [Description("The color of the light with a hex code (ensure you include the # symbol)")]
    public string? Color { get; set; }
}

[JsonConverter(typeof(JsonStringEnumConverter))]
public enum Brightness
{
    Low,
    Medium,
    High
}
```

> **Note**

While this is a "fun" example, it does a good job showing just how complex a plugin's parameters can be. In this single case, we have a complex object with *four* different types of properties: an integer, string, boolean, and enum. Semantic Kernel's value is that it can automatically generate the schema for this object and pass it to the AI agent and marshal the parameters generated by the AI agent into the correct object.

Once you're done authoring your plugin class, you can add it to the kernel using the `AddFromType<>` or `AddFromObject` methods.

> **Tip**
>
> When creating a function, always ask yourself "how can I give the AI additional help to use this function?" This can include using specific input types (avoid strings where possible), providing descriptions, and examples.

Adding a plugin using the `AddFromObject` method

The `AddFromObject` method allows you to add an instance of the plugin class directly to the plugin collection in case you want to directly control how the plugin is constructed.

For example, the constructor of the `LightsPlugin` class requires the list of lights. In this case, you can create an instance of the plugin class and add it to the plugin collection.

C#

```
List<LightModel> lights = new()
   {
      new LightModel { Id = 1, Name = "Table Lamp", IsOn = false, Brightness =
Brightness.Medium, Color = "#FFFFFF" },
      new LightModel { Id = 2, Name = "Porch light", IsOn = false, Brightness =
Brightness.High, Color = "#FF0000" },
      new LightModel { Id = 3, Name = "Chandelier", IsOn = true, Brightness =
Brightness.Low, Color = "#FFFF00" }
   };

kernel.Plugins.AddFromObject(new LightsPlugin(lights));
```

Adding a plugin using the `AddFromType<>` method

When using the `AddFromType<>` method, the kernel will automatically use dependency injection to create an instance of the plugin class and add it to the plugin collection.

This is helpful if your constructor requires services or other dependencies to be injected into the plugin. For example, our `LightsPlugin` class may require a logger and a light service to be injected into it instead of a list of lights.

C#

```
public class LightsPlugin
{
   private readonly Logger _logger;
   private readonly LightService _lightService;

   public LightsPlugin(LoggerFactory loggerFactory, LightService lightService)
   {
      _logger = loggerFactory.CreateLogger<LightsPlugin>();
      _lightService = lightService;
   }

   [KernelFunction("get_lights")]
   [Description("Gets a list of lights and their current state")]
   public async Task<List<LightModel>> GetLightsAsync()
   {
      _logger.LogInformation("Getting lights");
      return lightService.GetLights();
   }

   [KernelFunction("change_state")]
   [Description("Changes the state of the light")]
   public async Task<LightModel?> ChangeStateAsync(LightModel changeState)
   {
      _logger.LogInformation("Changing light state");
      return lightService.ChangeState(changeState);
   }
}
```

With Dependency Injection, you can add the required services and plugins to the kernel builder before building the kernel.

C#

```
var builder = Kernel.CreateBuilder();

// Add dependencies for the plugin
builder.Services.AddLogging(loggingBuilder =>
loggingBuilder.AddConsole().SetMinimumLevel(LogLevel.Trace));
builder.Services.AddSingleton<LightService>();

// Add the plugin to the kernel
builder.Plugins.AddFromType<LightsPlugin>("Lights");
```

```
// Build the kernel
Kernel kernel = builder.Build();
```

Defining a plugin using a collection of functions

Less common but still useful is defining a plugin using a collection of functions. This is particularly useful if you need to dynamically create a plugin from a set of functions at runtime.

Using this process requires you to use the function factory to create individual functions before adding them to the plugin.

C#

```
kernel.Plugins.AddFromFunctions("time_plugin",
[
    KernelFunctionFactory.CreateFromMethod(
        method: () => DateTime.Now,
        functionName: "get_time",
        description: "Get the current time"
    ),
    KernelFunctionFactory.CreateFromMethod(
        method: (DateTime start, DateTime end) => (end - start).TotalSeconds,
        functionName: "diff_time",
        description: "Get the difference between two times in seconds"
    )
]);
```

Additional strategies for adding native code with Dependency Injection

If you're working with Dependency Injection, there are additional strategies you can take to create and add plugins to the kernel. Below are some examples of how you can add a plugin using Dependency Injection.

Inject a plugin collection

> **Tip**
>
> We recommend making your plugin collection a transient service so that it is disposed of after each use since the plugin collection is mutable. Creating a new plugin collection for each use is cheap, so it should not be a performance concern.

C#

```
var builder = Host.CreateApplicationBuilder(args);

// Create native plugin collection
builder.Services.AddTransient((serviceProvider)=>{
   KernelPluginCollection pluginCollection = [];
   pluginCollection.AddFromType<LightsPlugin>("Lights");

   return pluginCollection;
});

// Create the kernel service
builder.Services.AddTransient<Kernel>((serviceProvider)=> {
   KernelPluginCollection pluginCollection =
serviceProvider.GetRequiredService<KernelPluginCollection>();

   return new Kernel(serviceProvider, pluginCollection);
});
```

> **Tip**
>
> As mentioned in the **kernel article**, the kernel is extremely lightweight, so creating a new kernel for each use as a transient is not a performance concern.

Generate your plugins as singletons

Plugins are not mutable, so its typically safe to create them as singletons. This can be done by using the plugin factory and adding the resulting plugin to your service collection.

C#

```
var builder = Host.CreateApplicationBuilder(args);

// Create singletons of your plugin
builder.Services.AddKeyedSingleton("LightPlugin", (serviceProvider, key) => {
    return KernelPluginFactory.CreateFromType<LightsPlugin>();
});

// Create a kernel service with singleton plugin
builder.Services.AddTransient((serviceProvider)=> {
    KernelPluginCollection pluginCollection = [
      serviceProvider.GetRequiredKeyedService<KernelPlugin>("LightPlugin")
    ];

    return new Kernel(serviceProvider, pluginCollection);
});
```

Providing functions return type schema to LLM

Currently, there is no well-defined, industry-wide standard for providing function return type metadata to AI models. Until such a standard is established, the following techniques can be considered for scenarios where the names of return type properties are insufficient for LLMs to reason about their content, or where additional context or handling instructions need to be associated with the return type to model or enhance your scenarios.

Before employing any of these techniques, it is advisable to provide more descriptive names for the return type properties, as this is the most straightforward way to improve the LLM's understanding of the return type and is also cost-effective in terms of token usage.

Provide function return type information in function description

To apply this technique, include the return type schema in the function's description attribute. The schema should detail the property names, descriptions, and types, as shown in the following example:

C#

```
public class LightsPlugin
{
   [KernelFunction("change_state")]
   [Description("""Changes the state of the light and returns:
   {
       "type": "object",
       "properties": {
           "id": { "type": "integer", "description": "Light ID" },
           "name": { "type": "string", "description": "Light name" },
           "is_on": { "type": "boolean", "description": "Is light on" },
           "brightness": { "type": "string", "enum": ["Low", "Medium", "High"], "de-
scription": "Brightness level" },
           "color": { "type": "string", "description": "Hex color code" }
       },
       "required": ["id", "name"]
   }
   """)]
   public async Task<LightModel?> ChangeStateAsync(LightModel changeState)
   {
      ...
   }
}
```

Some models may have limitations on the size of the function description, so it is advisable to keep the schema concise and only include essential information.

In cases where type information is not critical and minimizing token consumption is a priority, consider providing a brief description of the return type in the function's description attribute

instead of the full schema.

C#

```
public class LightsPlugin
{
   [KernelFunction("change_state")]
   [Description("""Changes the state of the light and returns:
        id: light ID,
        name: light name,
        is_on: is light on,
        brightness: brightness level (Low, Medium, High),
        color: Hex color code.
    """)]
   public async Task<LightModel?> ChangeStateAsync(LightModel changeState)
   {
      ...
   }
}
```

Both approaches mentioned above require manually adding the return type schema and updating it each time the return type changes. To avoid this, consider the next technique.

Provide function return type schema as part of the function's return value

This technique involves supplying both the function's return value and its schema to the LLM, rather than just the return value. This allows the LLM to use the schema to reason about the properties of the return value.

To implement this technique, you need to create and register an auto function invocation filter. For more details, see the Auto Function Invocation Filter article. This filter should wrap the function's return value in a custom object that contains both the original return value and its schema. Below is an example:

C#

```
private sealed class AddReturnTypeSchemaFilter : IAutoFunctionInvocationFilter
{
    public async Task OnAutoFunctionInvocationAsync(AutoFunctionInvocationContext
context, Func<AutoFunctionInvocationContext, Task> next)
    {
        await next(context); // Invoke the original function

        // Crete the result with the schema
        FunctionResultWithSchema resultWithSchema = new()
        {
            Value = context.Result.GetValue<object>(),                 // Get the
original result
```

```
            Schema = context.Function.Metadata.ReturnParameter?.Schema  // Get the
function return type schema
        };

        // Return the result with the schema instead of the original one
        context.Result = new FunctionResult(context.Result, resultWithSchema);
    }

    private sealed class FunctionResultWithSchema
    {
        public object? Value { get; set; }
        public KernelJsonSchema? Schema { get; set; }
    }
}

// Register the filter
Kernel kernel = new Kernel();
kernel.AutoFunctionInvocationFilters.Add(new AddReturnTypeSchemaFilter());
```

With the filter registered, you can now provide descriptions for the return type and its properties, which will be automatically extracted by Semantic Kernel:

C#

```
[Description("The state of the light")] // Equivalent to annotating the function with
the [return: Description("The state of the light")] attribute
public class LightModel
{
    [JsonPropertyName("id")]
    [Description("The ID of the light")]
    public int Id { get; set; }

    [JsonPropertyName("name")]
    [Description("The name of the light")]
    public string? Name { get; set; }

    [JsonPropertyName("is_on")]
    [Description("Indicates whether the light is on")]
    public bool? IsOn { get; set; }

    [JsonPropertyName("brightness")]
    [Description("The brightness level of the light")]
    public Brightness? Brightness { get; set; }

    [JsonPropertyName("color")]
    [Description("The color of the light with a hex code (ensure you include the #
symbol)")]
    public string? Color { get; set; }
}
```

This approach eliminates the need to manually provide and update the return type schema each time the return type changes, as the schema is automatically extracted by the Semantic Kernel.

Next steps

Now that you know how to create a plugin, you can now learn how to use them with your AI agent. Depending on the type of functions you've added to your plugins, there are different patterns you should follow. For retrieval functions, refer to the using retrieval functions article. For task automation functions, refer to the using task automation functions article.

Learn about using retrieval functions

Last updated on 03/07/2025